

Aspects of Flow Matching

Amey Gaikwad, Sven Harder

UCLA

E-mail: ameypg@physics.ucla.edu, sharder@physics.ucla.edu

ABSTRACT: In this note, we will summarize our work on the flow matching, using an induced metric. We construct a flow model that generates flows constrained to a simple submanifolds embedded in three dimensions. For this, we first train a KMeans model to construct a metric on the data and use two neural networks to first learn the geodesics of the metric and the the geodesic flow vector field that generates them. We find success on transporting data on genus zero, one and two surfaces and look forward to apply this framework to more complicated datasets in the future.

Contents

1	Big Picture	2
2	Getting the Data	2
3	Exploring and Preparing the Data	3
4	The Model	5
4.1	Learning a Metric	5
4.2	Networks for Flow Matching	7
4.2.1	Interpolant Network	7
4.2.2	Flow Network	8
5	Finetuning the Model	8
6	Our Solution	11

1 Big Picture

The central question of this project is whether *metric aware flow matching* can learn transport trajectories that stay on a data manifold, rather than taking straight line shortcuts in ambient Euclidean space. Standard flow matching methods are effective for distribution transport, but they do not explicitly enforce geometric or topological structure of the data. For manifold supported data, this can produce unrealistic intermediate states. This problem has been discussed in many papers. We orient our discussion along the recent [1].

Our objective is to build and test a geometry aware transport pipeline on controlled synthetic manifolds in $\mathbb{R}^3$: a sphere, a torus, and a genus 2 surface. These three datasets provide increasing geometric complexity, so they serve as a useful benchmark for whether the model respects curvature and topology during transport.

We frame this as an offline learning problem with paired source and target samples. The method has three coupled stages:

1. Learn a metric from the data
2. Train an interpolant network to produce low energy paths under that metric
3. Train a velocity network to match those interpolants and generate flows

This structure is motivated by prior metric flow matching work, but we implement our own pipeline and datasets.

Current workarounds include Euclidean interpolation and unconstrained flow models, which often ignore manifold structure. In contrast, we use metric weighted objectives so the optimization directly favors geometry consistent motion.

We judge success using both quantitative and qualitative criteria. Quantitatively, we monitor the metric weighted interpolant and velocity losses. Qualitatively, we check whether trajectories remain manifold consistent and whether transported samples align with target modes. Minimum success means stable training, plausible manifold respecting paths, and reasonable endpoint overlap.

A key assumption is that the learned RBF metric is expressive enough to capture useful geometry for transport. Another is that synthetic manifold datasets are a valid stress test for topology aware generative flow methods.

2 Getting the Data

Our project does not use an external dataset. Instead, we generate all data locally in code so we can fully control geometry, topology, and noise. We construct paired source/target point clouds in $\mathbb{R}^3$ for three manifolds:

1. Sphere
2. Torus
3. Genus-2 surface

For each manifold, we typically use $n \approx 3500$ points per source and target set for visualization/debugging, and up to $n \approx 4000$ per set for training runs. The generation is deterministic under fixed random seeds, which makes experiments reproducible across reruns.

The sampled data are first created as NumPy arrays and then converted to PyTorch tensors in `float32` format for training (including GPU compatible workflows). Storage requirements are small (point clouds + model checkpoints), so all experiments fit comfortably in local project storage.

Since the data is synthetic, there are no licensing, privacy, or access authorization constraints, and no sensitive information handling is required. The datasets are 3D point cloud data with manifold support, not time series or tabular records.

Because the data set is generated on demand, evaluation sets can be created by reserving independent seeds. In the current workflow, we primarily evaluate via training/evaluation losses and qualitative transport diagnostics, but strict train/test separation is straightforward to add.

3 Exploring and Preparing the Data

Each dataset is represented as a 3D point cloud with coordinates (x, y, z) for source samples x_0 and target samples x_1 . There are no missing values, and all coordinates are continuous `float32` features bounded by the geometry construction. The dominant variability is intentional sampling noise (core/tail spread, angular perturbations, thickness noise), not measurement corruption.

- **Attribute summary :** The three coordinate dimensions are all essential for the task. Their distributions are non-Gaussian and manifold supported, so linear feature assumptions are generally not appropriate. This is not a standard class label supervised setting; supervision is objective-based:
 - the interpolant network is trained by a metric weighted energy loss,
 - the velocity network regresses to interpolant derivatives,
 - endpoint transport is evaluated against the target cloud x_1 .
- **Dataset specific observations:**
 - **Sphere:** source and target are concentrated near opposite poles with visible tails toward intermediate latitudes.
 - **Torus:** source/target modes sit on opposite sides of a periodic geometry; valid paths should follow toroidal structure.
 - **Genus-2:** two-handle topology with bridge support.
- **Correlation structure :** Correlations are primarily geometric and nonlinear,
 - sphere data approximately satisfy $x^2 + y^2 + z^2 \approx 1$,

- torus data approximately satisfy $(\sqrt{x^2 + y^2} - R)^2 + z^2 \approx r^2$,
- genus 2 data show region dependent local structure (handles vs. bridge).

Manual baseline intuition: A manual approach would route trajectories along intrinsic manifold directions: great circles on the sphere, cycle respecting motion on the torus, and handle/bridge respecting motion on genus-2. This motivates metric-aware transport objectives.

Main takeaway from exploration: As manifold complexity increases (sphere to torus to genus 2), the risk of unrealistic ambient space shortcuts increases. This directly motivates the use of a learned metric and metric weighted losses in later sections.

In fig. 1 we plot the initial (blue) and final (red) data distributions on the three manifolds considered. What we concretely wish to do is learn a metric on the data manifold and then produce geodesics on this data manifold, which in turn serve as curved interpolants connecting the two data distributions.

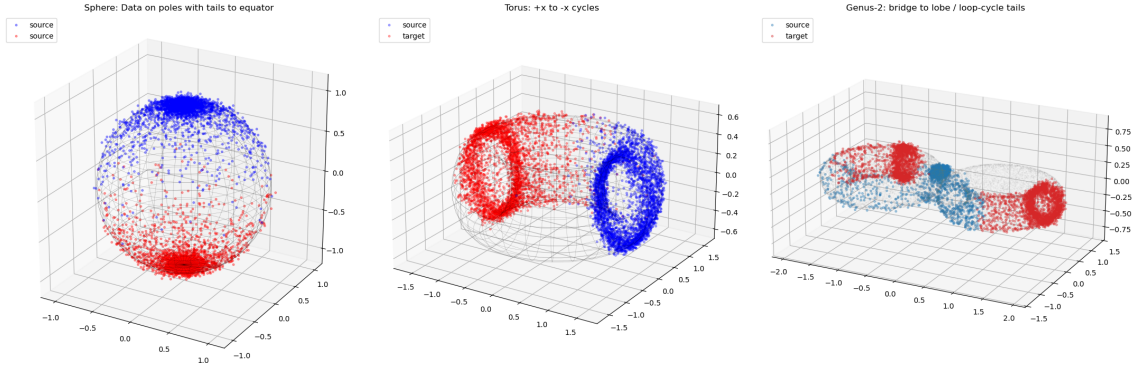


Figure 1. Visualization of the initial (blue) and final (red) data distributions on the three manifolds considered.

4 The Model

In model implementation, we follow [1]. The authors have three steps that include machine learning techniques:

1. Learning a metric $\mathbf{G}$, using KMeans and an MSE loss function.
2. Learning interpolants that are approximate geodesics of G , using a fully connected neural network ϕ_θ .
3. Learning the flow field, generating the geodesics v_θ .

The performance of each network strongly relies on good performance of the previous ones. We will explain each step in detail below.

4.1 Learning a Metric

Given any finite set of data points in high dimensional space, there is an infinite number of ways of assigning a metric on these points. A priori, there is no preferred choice. A *good* choice for our goal is one that is large on data points and vanishes away from data points. Examples have been studied in [2]. We will implement a simple one that is called the RBF metric. It is obtained in a two step procedure:

1. Use KMeans clustering to identify centers and add Gaussians with width given by the standard deviation.
2. Assign additional weights to each Gaussian, such that the metric is approximately one at each data point.

More mathematically:

- We take our metric to be diagonal with:

$$\mathbf{G} = (\text{diag}(\vec{h}) + \epsilon \mathbf{1})^{-1} \quad (4.1)$$

where ϵ is a regularizer to make the metric locally invertible.

- The vector $\vec{h}$ is a sum of weighted Gaussians of the form:

$$h_\alpha(x) = \sum_{k=1}^K \omega_{\alpha,k}(x) \exp\left(-\frac{\lambda_{\alpha,k}}{2} |x_\alpha - \hat{x}_{\alpha,k}|^2\right) \quad (4.2)$$

- We first run a clustering algorithm to find:

1. Clusters C_k with positions $\hat{x}_k$

2. Corresponding bandwidths $\lambda_{\alpha,k}$ defined as:

$$\lambda_{\alpha,k} = \frac{1}{2} \left(\frac{\kappa}{|C_k|} \sum_{x \in C_k} |x_\alpha - \hat{x}_{\alpha,k}|^2 \right)^{-2} \quad (4.3)$$

These should be thought of as inverse widths of the center. The hyperparameter κ allows for an additional squeezing. We did not find it necessary to use it

- Determine the weight functions $\omega_{\alpha,k}$ by minimizing the MSE error:

$$\mathcal{L} = \sum_{x \in \mathcal{D}} (\vec{1} - \vec{h}(x))^2 \quad (4.4)$$

This regresses the metric to be uniform on the data points.

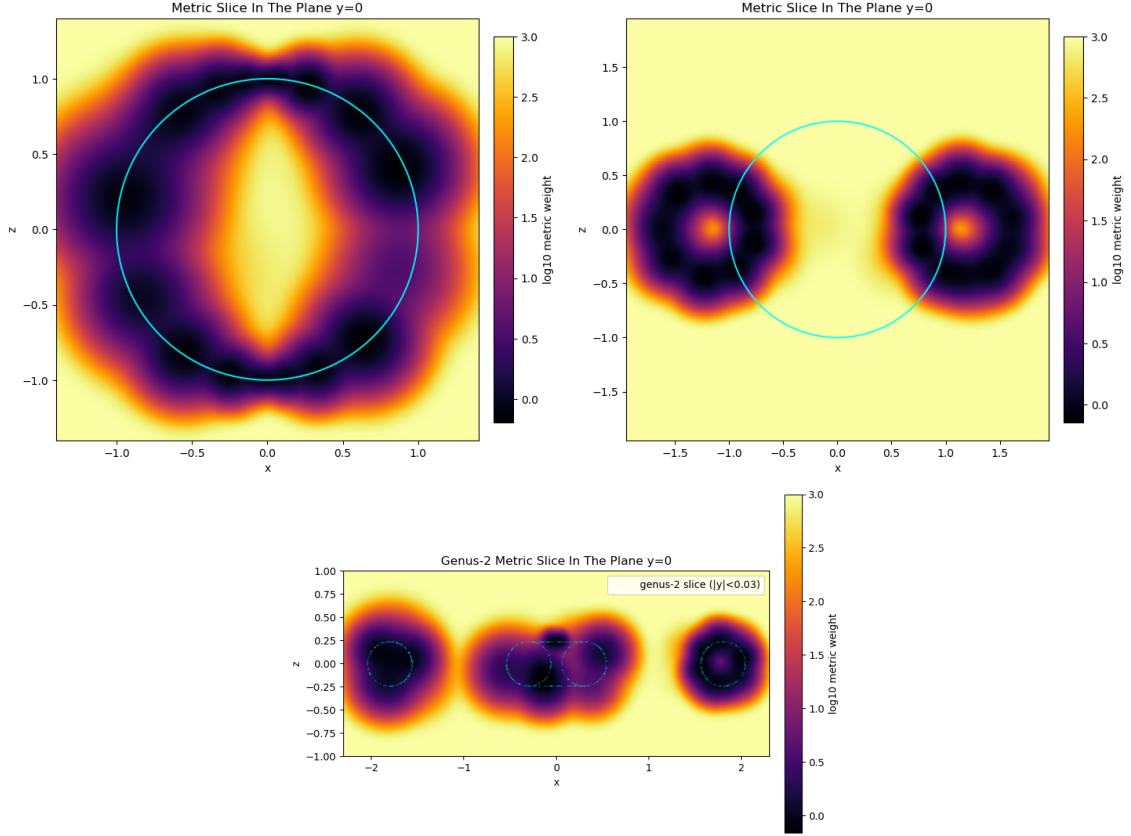


Figure 2. Visualization of the metric given the data distributions in fig. (1). Clearly we see that the metric has non trivial support on the data manifold itself !

There are other metrics that are described in both papers. They usually have a hyperparameter σ similar to κ . This hyperparameter requires optimization. While the bandwidth characterize length scales on the data manifold σ is the only scale in the LAND metric. We did not find it instructive to use it.

In fig. 2, we present the visualizations of the metric defined on the data manifold given in fig. 1. There are multiple important comments to make here:

- Changing the hyperparameter κ would make the metric get more fizzled out. Then, the entire topological structure would get lost.
- In these runs, we choose $n = 60$. This was a good compromise of computational time and accuracy. When increasing the number of cnetroids, the second optimization step for the weighting functions tended to take longer.
- There are slightly degenerate cases on manifolds, where there are multiple geodesic solutions. E.g. between the north and south pole on a sphere, all great circles are valid solutions to the equations. The way we construct the metric from the data will wash out such degeneracies and will give the problem a unique solution.
- If one would not have access to any tails of the data or some more or less complete trajectory in data space, this method is useless. The metric would not prefer any obvious path between the centers of the data.

4.2 Networks for Flow Matching

As described above, the flow matching takes place in a two step process:

1. Train interpolants ϕ_θ
2. Train velocity field v_θ

For both of them, we are using fully connected neural networks, we will describe their basic architecture below.

4.2.1 Interpolant Network

In standard flow matching, one samples pairs $(\vec{x}_0, \vec{x}_1)$ from initial and final distribution and connects them with a linear interpolant:

$$\vec{x}(t) = (1 - t)\vec{x}_0 + \vec{x}_1 t \quad (4.5)$$

The network then regresses to the time derivative of these. To learn different trajectories, we first need to make sure that we have different interpolants. These are now implemented by a neural net:

$$\vec{x}(t) = (1 - t)\vec{x}_0 + \vec{x}_1 t + t(1 - t)\phi_\theta \quad (4.6)$$

where θ are the network parameters. It is a map:

$$\begin{aligned} \phi_\theta : \quad \mathbb{R}^7 &\mapsto \mathbb{R}^3 \\ (\vec{x}_0, \vec{x}_1, t) &\mapsto \phi(t) \end{aligned} \quad (4.7)$$

that gives the contribution to the position at t for given boundary conditions.

The key trick is to minimize this to learn geodesics. The action principle that geodesics minimize can be written as:

$$S = \int_{\vec{x}_0}^{\vec{x}_1} g_{ij}(x(t)) \dot{x}^i(t) \dot{x}^j(t) dt \quad (4.8)$$

The simplest discretization for our purposes is:

$$\mathcal{L} = \sum_{t_i} \dot{\vec{x}}^T(t_i) \mathbf{G}(\vec{x}(t_i)) \dot{\vec{x}}(t_i) \quad (4.9)$$

where t_i are sampled timesteps and $\vec{x}(t_i)$ depends on ϕ . Some points that are important to bring up are:

- In our algorithm, we sampled $(\vec{x}_0, \vec{x}_1)$ from the product distribution. Generically, one can decide on any joint distribution $\Pi(\vec{x}_0, \vec{x}_1)$ with the correct marginals. In [1] they implement an additional Wasserstein distance for this.
- This training is averaging over the data. The resulting trajectories may not be the geodesics that do solve the given boundary value problem for all pairs. They are optimized to fit the average.
- In general any pairing of data in this example is random. If one knows which data points will evolve into each other, there are more interesting constraints, one can put.

4.2.2 Flow Network

The flow network is identical to standard flow networks as first introduced in [3]. It tries to regress to the interpolants. It is of the form:

$$\begin{aligned} v_\theta : \quad \mathbb{R}^4 &\mapsto \mathbb{R}^3 \\ (\vec{x}, t) &\mapsto v_\theta(\vec{x}, t) \end{aligned} \quad (4.10)$$

Its loss is the MSE to the interpolant:

$$\mathcal{L} = \sum_{(\vec{x}_0, \vec{x}_1), t_i} (v_\theta(\vec{x}(t_i), t_i) - \dot{\vec{x}}(t_i))^T \mathbf{G}(\vec{x}(t_i)) (v_\theta(\vec{x}(t_i), t_i) - \dot{\vec{x}}(t_i)) \quad (4.11)$$

The weighting by the metric is not necessary, but it is useful, because it puts additional weight on points close to data points that are preferred by the metric.

5 Finetuning the Model

The hyperparameter optimization in this model is slightly different from standard hyperparameter optimization:

- We only want to employ the velocity network v_θ after training to push forward data points. Thus actual performance of the entire algorithm will be measure by the performance of the final network on evaluation.

- In the case of simple manifolds, there are very obvious evaluation metrics for the first two parts. We could compare to the original metrics and geodesics of the manifolds. We decided against employing this measure. If we would like to apply this algorithm to an abstract data distribution, we do not have access to such evaluation metrics. Thus it is not particularly useful in the case of actually interesting problems.
- The best evaluation measure for the velocity network would be comparing the final distribution against the pushed forward initial distribution by a statistical measure like the Wasserstein distance. This was not necessary in the case of these simple examples. For more complicated examples, it would prove helpful.
- Instead of employing more advanced metrics, we use cross validation by picking out a batch from the training set randomly and evaluating the loss functions of the given networks on it. We find success with this strategy.

The training pipeline is stage coupled rather than a single end to end predictor. Instead, we first learn the metric, then train the interpolant network ϕ_θ , and only after that train the velocity network v_ψ . Since v_ψ is trained to match derivatives produced by ϕ_θ , errors in the metric or interpolant stage directly affect everything that follows.

For this reason, we use a sequential tuning strategy. We first tune ϕ_θ , select the best checkpoint, and freeze it. We then tune v_ψ conditioned on that fixed interpolant. This differs from standard workflows where all hyperparameters are searched jointly on one model family. In our setting, tuning order matters: a weak interpolant can make velocity tuning look poor even when velocity hyperparameters are reasonable.

We also use a two stage search budget for each network. The first stage is a broad coarse scan with short training runs to identify promising regions of the search space. The second stage retrains the top k candidates with a longer budget and selects the best final model. In practice, this gives better stability than choosing a model from short runs alone, especially on the torus and genus 2 datasets.

The main tuned hyperparameters are hidden width, number of hidden layers, learning rate, and batch size. We evaluate candidates with the stage aligned losses: metric weighted interpolant loss for ϕ_θ and metric weighted velocity matching loss for v_ψ . We also verify qualitative behavior using trajectory plots (see figure 3), because low loss alone does not guarantee topology respecting transport.

Overall, finetuning confirms that model performance is constrained by upstream quality. Better metric and interpolant fits produce more stable velocity learning and cleaner transport trajectories. This coupling is the key reason our optimization procedure is more structured than standard hyperparameter tuning.

Table 1. We tabulate the optimal network hyperparameters selected by staged tuning.

Dataset	Network	Width of NN	Hidden layers	Learning rate	Batch size	Steps
Sphere	ϕ_θ	128	4	1×10^{-3}	128	3000
Sphere	v_ψ	128	4	3×10^{-4}	128	5000
Torus	ϕ_θ	256	4	3×10^{-4}	256	3000
Torus	v_ψ	128	4	1×10^{-3}	256	5000
Genus-2	ϕ_θ	256	6	3×10^{-4}	256	3000
Genus-2	v_ψ	256	6	1×10^{-3}	64	5000

6 Our Solution

Our final solution is a metric aware flow matching pipeline that combines three learned components: an RBF metric model, an interpolant network, and a velocity network. The metric model provides a geometry sensitive weighting over ambient space, so motion near manifold supported regions is favored during optimization. The interpolant network then learns smooth paths between source and target samples under this learned metric, and the velocity network is trained to reproduce the interpolant derivatives so that we can integrate flows forward in time.

This structure allows us to move beyond Euclidean straight line interpolation and instead produce transport trajectories that are consistent with manifold geometry and topology. We tested the same pipeline on sphere, torus, and genus 2 datasets to evaluate robustness across increasing complexity. On the sphere, trajectories remain near the spherical shell and connect opposite modes smoothly. On the torus, transport follows toroidal structure instead of cutting through low density ambient regions. On the genus 2 surface, the model can split and route trajectories through valid handle/bridge regions before converging toward disjoint target modes.

The strongest practical result is that the method is effective when all stages are well trained. In other words, success depends on the full chain: metric quality affects interpolant quality, and interpolant quality affects velocity fitting. With the tuned configurations from our staged hyperparameter search, we obtained stable optimization and visually consistent transport behavior across all three geometries, with genus 2 requiring the most capacity and tuning effort.

In summary, our solution demonstrates that metric aware flow matching is a viable approach for geometry constrained transport on nontrivial manifolds. The experiments show that adding a learned metric prior and stage wise training improves manifold adherence of trajectories compared with unconstrained ambient space interpolation.

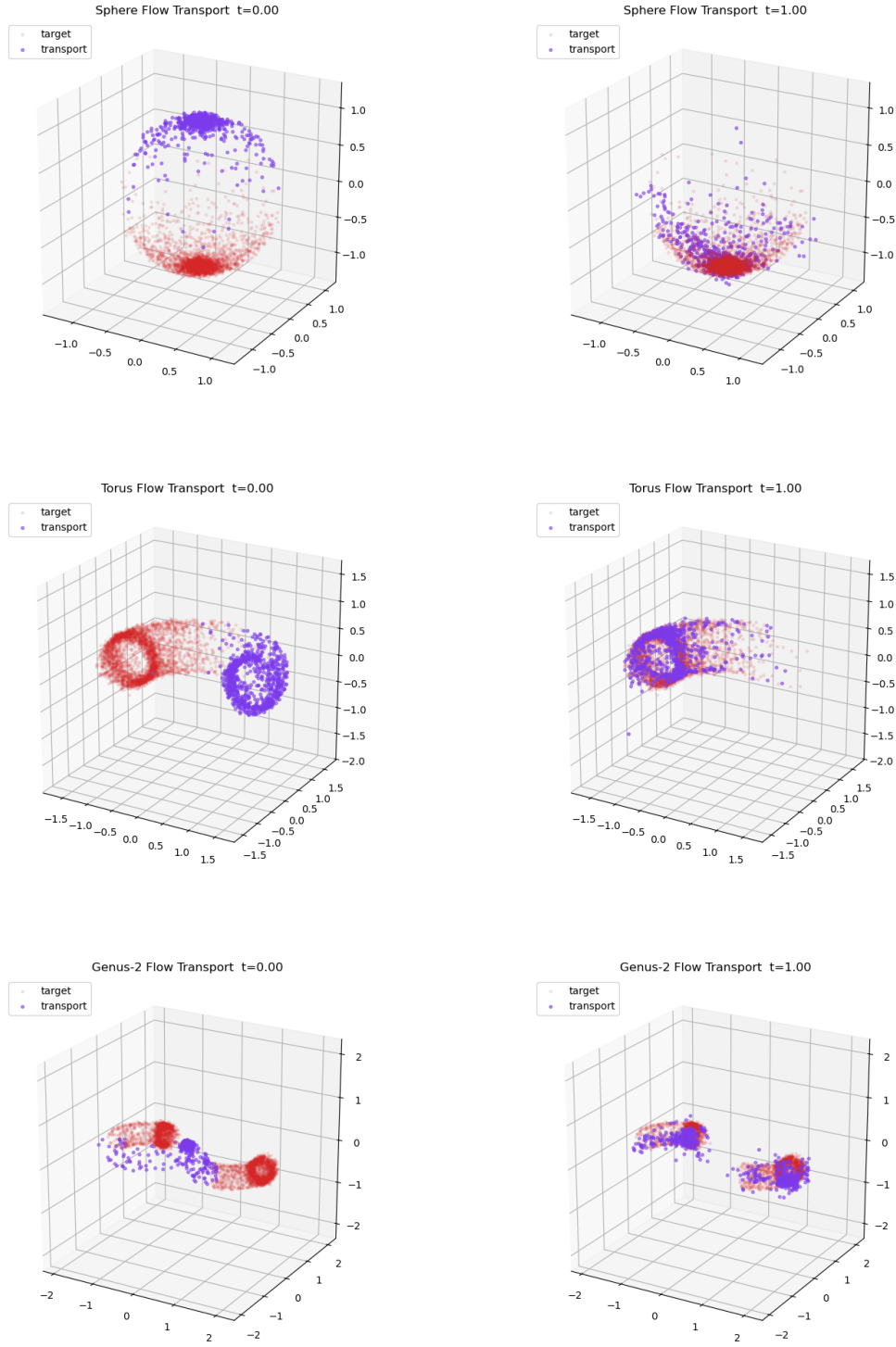


Figure 3. Snapshots of the learned flow on the three manifolds considered. For each row, the left panel shows the distribution at the initial time and the right panel shows the distribution at a later stage of the transport, with initial samples shown in purple and target samples in red. From top to bottom, the rows correspond to the sphere, torus, and genus-2 surface, respectively.

References

- [1] K. Kapuśniak, P. Potapchik, T. Reu, L. Zhang, A. Tong, M. Bronstein et al., *Metric flow matching for smooth interpolations on the data manifold*, [2405.14780](#).
- [2] G. Arvanitidis, S. Hauberg and B. Schölkopf, *Geometrically enriched latent spaces*, [2008.00565](#).
- [3] Y. Lipman, R.T.Q. Chen, H. Ben-Hamu, M. Nickel and M. Le, *Flow matching for generative modeling*, [2210.02747](#).